

PQ-Shield: An Empirical Framework for Security–Performance Benchmarking of Post-Quantum Cryptography in Real-Time and Streaming AI Inference APIs

Nivitha K

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Manav Prakash

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Shashwat Kumar

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Yatharth Srivastava

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Abstract—AI inference APIs are protected almost entirely by classical key exchange and signatures, which a cryptographically relevant quantum computer (CRQC) would break retroactively: traffic harvested today can be decrypted later. We present PQ-Shield, a benchmarking framework that wraps a real FastAPI inference service in six configurations—an unprotected control, legacy RSA key transport, production-style X25519, ML-KEM-768, the deployed X25519MLKEM768 hybrid, and ML-KEM-768 with ML-DSA-65 signatures—and measures them under concurrency (10–1,000 connections), four payload shapes, emulated network paths, session reuse, and active and passive attacks, with streaming responses from a real Llama-3.2-3B model. Against the realistic X25519 baseline, post-quantum key establishment adds no measurable latency: every non-RSA configuration, X25519 included, is within +3.3–7.8% of the control at 10 connections; it costs about 5 ms on a 5 Mbit/s link, and its residual overhead falls to 0–2% when one key exchange serves 100 requests. Per-handshake RSA-2048 key generation, by contrast, occupies 4.8 CPU cores at 10 connections and nearly doubles latency. For token streaming we treat per-chunk and hash-chained signing as endpoints of one design space—a hash chain checkpointed every k chunks—and measure its trade-off between signature bytes and exposure, the tokens shown before verification: with ML-DSA-65 a 200-token response carries 135,669 B of signatures at $k=1$ and 3,309 B at $k=\infty$, and checkpoints move detection of dropped or reordered chunks from the end of the stream to the next checkpoint. Under classical key exchange a future CRQC could decrypt 11.4 bytes per streamed token; under ML-KEM, none. All post-quantum primitives pass NIST ACVP known-answer tests (75/75).

Index Terms—post-quantum cryptography, ML-KEM, ML-DSA, hybrid key exchange, harvest-now-decrypt-later, AI inference APIs, LLM streaming, stream authentication, benchmarking

I. Introduction

AI inference endpoints—hosted by commercial providers, on managed cloud platforms, or as custom FastAPI/Flask services—carry model inputs and outputs that can be sensitive: diagnostic features, financial records, and increasingly the full text of conversational and agentic sessions. Inference endpoints are already a recognised attack surface for interception, tampering, and privacy leakage [10], [11]. Their transport protection rests on RSA or elliptic-curve key exchange and signatures, all of which Shor’s algorithm breaks in polynomial time on a sufficiently large fault-tolerant quantum computer [21]. Such a machine does not yet exist, but the harvest-now-decrypt-later (HN DL) threat means an adversary can record encrypted traffic today and decrypt it once one does; data whose confidentiality must outlive the arrival of a CRQC is already exposed [13], [22].

NIST finalized its first post-quantum standards in August 2024: ML-KEM (FIPS 203) for key encapsulation [16] and ML-DSA (FIPS 204) for signatures [17]. Protocol-level migration is under way—the X25519MLKEM768 hybrid group is already negotiated by major browsers and content-delivery networks [20]—and post-quantum TLS has been benchmarked extensively [6], [14], [24], [25]. What is missing is an empirical account of what migration costs an AI inference workload specifically: small structured requests, inference times from milliseconds to seconds, and, for large language models, responses that are streamed token by token and therefore cannot be signed as a whole before the first byte is sent.

This paper makes seven contributions. First, a concurrency benchmark (10–1,000 connections, five repetitions per cell) of six configurations on a real classifier workload, including a production-style X25519 baseline and the deployed X25519MLKEM768 hybrid, with server CPU and memory. Second, a payload-shape study across four AI workload profiles (163 B–8 kB). Third, network-emulation and session-reuse experiments that isolate the cost of post-quantum message size and quantify how session reuse amortizes key establishment. Fourth, passive (HN DL) and active (tampering) threat experiments integrated into the pipeline. Fifth, an adaptation of amortized stream signing [26]–[28] to token-by-token LLM responses on a real Llama-3.2-3B backend, where a post-quantum signature’s size (3,309 B for ML-DSA-65 versus ≈ 71 B for ECDSA P-256) makes per-chunk signing about forty times costlier in bytes; we show that naive per-chunk signing admits an undetected reordering attack and close it by binding the chunk index into the signed bytes. Sixth, we treat per-chunk and hash-chained signing as the two endpoints of one design space—a hash chain checkpointed every k chunks—define exposure, the number of tokens a client displays before any signature covers them, and measure signature bytes, exposure, and mid-stream attack detection for $k \in \{1, 2, 5, 10, 20, \infty\}$ against an analytic model that the measurements match exactly. Seventh, ground-truth validation of the post-quantum bindings against NIST ACVP known-answer tests and of the streaming harness against an analytic model.

II. Background and Related Work

A. Post-Quantum Primitives

ML-KEM-768 (FIPS 203) is a module-lattice key-encapsulation mechanism at NIST security category 3, with public key, secret key, and ciphertext sizes of 1,184, 2,400, and 1,088 bytes and a 32-byte shared secret. ML-DSA-65 (FIPS 204) is a module-lattice signature scheme at the same category, with public key, secret key, and signature sizes of 1,952, 4,032, and 3,309 bytes. Several publications that post-date FIPS 204 still quote pre-standardization CRYSTALS-Dilithium3 values (a 3,293-byte signature and 4,000-byte secret key); all values in this paper are confirmed against FIPS 204 as finalized (Section VII). The X25519MLKEM768 hybrid [20] concatenates an ML-KEM-768 share with a 32-byte X25519 share and derives the session key from both shared secrets, so it stays secure unless both algorithms are broken.

B. Related Work

Table I synthesizes fifteen studies spanning heterogeneous-hardware PQC benchmarking, industry deployment, layered TLS 1.3 evaluation, migration surveys, 5G/6G integration, ML threat modeling, and HN DL economics. None combines AI-specific payload profiling, application-layer load sweeps, and integrated adversarial experiments, and none addresses streaming

response signatures. Three further lines of work bear directly on our design.

Post-quantum TLS under network conditions: Paquin et al. [24] benchmarked post-quantum key exchange and signatures inside TLS 1.3 over emulated links (Linux netem), showing that on lossy or low-bandwidth paths message size, not computation, dominates handshake latency. Sikeridis et al. [25] reached a similar conclusion for post-quantum authentication, and Montenegro et al. [6] and Gómez-Cambronero et al. [14] extend the analysis to layered TLS and application-level load. These works measure the TLS handshake with generic payloads. PQ-Shield instead measures an application-layer protocol around AI inference payloads, including streaming responses, and adds controlled adversarial experiments. Our network emulation (Section V-H) is deliberately coarser than theirs—it shapes the byte stream rather than packets—and we use it to bound, not replace, their packet-level results.

Stream authentication: Signing a stream that is not available in full when transmission starts is a classical problem. Gennaro and Rohatgi [26] chain per-block hashes so that one signature authenticates an entire stream; Wong and Lam [27] amortize one signature over a flow with hash trees; TESLA and EMSS [28] trade immediate verification for low per-packet overhead. Our hash-chain strategy is an instance of this family, not a new construction, and inherits the same trade-off: integrity is confirmed only when a signature arrives. What is new is the setting. LLM token streams have a small, latency-critical first chunk and, with post-quantum signatures, a per-signature cost large enough that the choice of amortization decides whether streaming is affordable at all; we make that choice explicit and measurable through checkpointing (Section V-F).

Side channels in encrypted LLM streams: Weiss et al. [29] recovered the content of encrypted AI-assistant responses from the sizes of streamed packets (the token-length side channel). The attack is orthogonal to the key exchange and applies to every protected configuration here, but it sharpens one qualification: per-chunk and hash-chained signing emit one record per chunk and therefore expose chunk count and timing, which only buffer-and-sign hides.

III. Threat Model

A. Harvest-Now-Decrypt-Later (HN DL)

A passive adversary records every cryptographic artifact on a protected connection—the key-establishment blob, every AES-256-GCM (nonce, ciphertext) pair, and any signatures—without decrypting anything. The AES-256-GCM layer is not considered CRQC-decryptable in any configuration (Grover’s algorithm only halves its effective key length), so what differs between configurations is whether the key-establishment blob, and with it the session key, becomes recoverable once a CRQC

TABLE I
Synthesis of related work and the gap each leaves open

Study	Focus	Limitation Addressed Here
Abbasi et al. [1]	Heterogeneous-hardware PQC benchmark	No AI workload or network stack
Demir et al. [2]	Industry TLS/telecom PQC deployment	No application-layer API metrics
Egbuagha & Ikwunna [3]	Protocol-level PQC readiness review	No experimental data
Pote & Bansode [4]	Micro-benchmark of core PQC operations	No web protocol context
Shaw [5]	Python X25519+ML-KEM hybrid library	No adversarial threat model
Montenegro et al. [6]	Post-quantum TLS 1.3 evaluation framework	Restricted to generic TLS traffic
Chhetri et al. [7]	Quantum-safe migration survey	Qualitative; no performance data
Faval et al. [8]	PQC in 5G core networks	Telecom-specific profiles
Kudaloor & Aijaz [9]	PQC for 6G wireless nodes	No REST/JSON API pipeline
Song et al. [10]	Privacy risks of robust ML models	Pre-dates practical PQC standards
Tidjon & Khomh [11]	Threat assessment for ML systems	Ignores transport-layer crypto
Ahmed et al. [12]	PQC support in crypto libraries	No end-to-end measurement
Blanco-Romero et al. [13]	Economic feasibility of HNDL	No application-layer context
Gómez-Cambronero et al. [14]	Layered TLS 1.3 handshake performance	Fixed key exchange; no signature sweep
Schemitt et al. [15]	PQC signatures on blockchains	Domain-restricted to blockchain

exists. RSA-OAEP and X25519 do; ML-KEM-768 and X25519MLKEM768 do not.

B. Active Man-in-the-Middle (MITM)

An active adversary on the wire intercepts a response and flips one byte, either inside the AEAD ciphertext or inside the signature, and we observe whether and where the client rejects it. A ciphertext flip should be rejected by the AEAD authentication-tag check before the payload reaches the application; a signature flip is caught only when the signature is verified. We log the detection layer and the time of the rejecting check itself, which separates configurations that merely detect tampering from those that detect it cheaply and early.

C. Streaming-Specific Threats

Two threats appear only when one connection streams many chunks. The first is a sequence-integrity attack: the adversary drops or reorders chunks without altering any chunk’s bytes. Each chunk’s AEAD tag and, under naive per-chunk signing, its signature remain valid, so detection requires binding chunks to an explicit position; we measure which strategies do so and how early in the stream they detect the attack.

The second is HNDL exposure that compounds with session length. A streamed session reuses one session key for every chunk, so a passive adversary who records the handshake captures the whole session at once: the longer the stream, the more ciphertext becomes retroactively readable the moment the key exchange is broken. We quantify this by relating the number of generated tokens to the volume a single future key recovery would expose. None of the evaluated configurations rotates keys within a session; mid-stream rekeying is left to future work.

IV. System Design

Six processes wrap the same FastAPI inference server (Fig. 1): an unprotected control and five protected configurations (Table II) sharing a common ServerCryptoConfig/ClientCryptoConfig contract, so that only the

TABLE II
Configurations. ECDSA is over P-256; “PQ”: key exchange resists a CRQC. All protected configurations share AES-256-GCM with an HKDF-SHA256 session key.

Configuration	Key establishment	Signature	PQ
Control	none	none	—
A: Classical-RSA	RSA-2048-OAEP (new key per handshake)	ECDSA	no
A’: Classical-ECDHE	ephemeral X25519	ECDSA	no
B: Hybrid	ML-KEM-768	ECDSA	yes
B’: Hybrid-KEX	X25519MLKEM768	ECDSA	yes
C: Full PQC	ML-KEM-768	ML-DSA-65	yes

asymmetric key-establishment and signature primitives differ. All protected configurations use the same AES-256-GCM layer keyed by HKDF-SHA256 over the raw shared secret. Every protected transaction follows the same five steps—handshake, session-key establishment, encrypt-and-send, decrypt, verify (Fig. 2)—with a fresh ephemeral key pair per transaction by default.

Configuration A’ is the production classical baseline: TLS 1.3 removed RSA key transport [19], and ephemeral X25519 is the classical half of the X25519MLKEM768 group now negotiated on the web [20]. A’ and B have identical signature code, so A’ → B isolates exactly one change, X25519 to ML-KEM-768. Configuration A generates a fresh RSA-2048 key pair per handshake; no production system does this, so we report A as a legacy worst case that exposes the cost of RSA key generation under load, not as the classical baseline. Configuration B pairs a post-quantum key exchange (ML-KEM-768 alone) with a classical signature. Configuration B’ instead combines X25519 and ML-KEM-768 within the key exchange, following [20]: the server’s share is the ML-KEM encapsulation key followed by an X25519 public key (1,216 B), the client’s is the ML-KEM ciphertext followed by its X25519 public key (1,120 B), and the session key is derived from both shared secrets concatenated. With

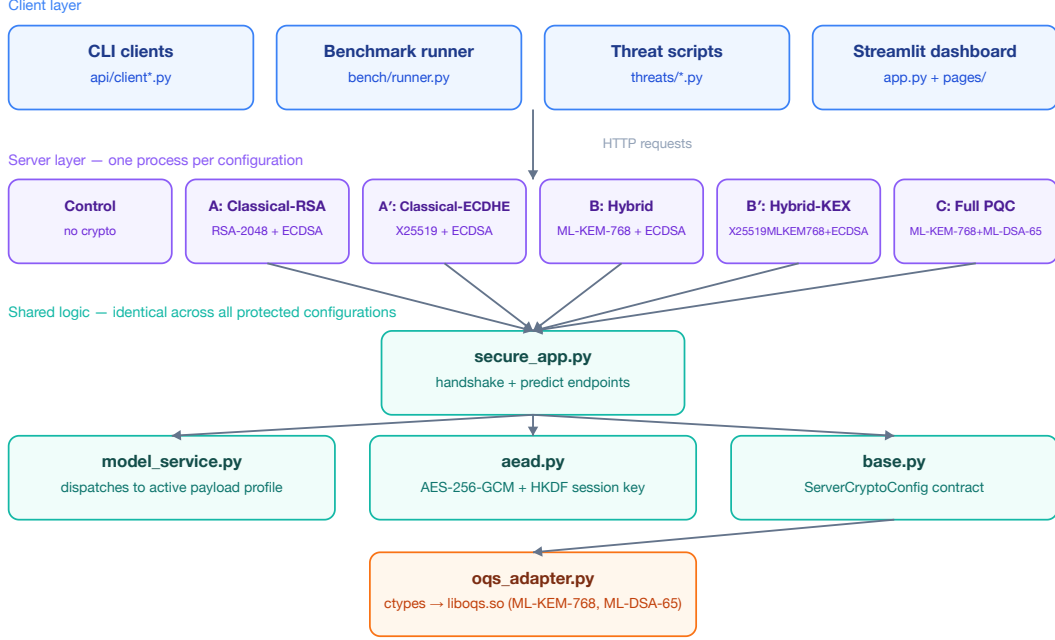


Fig. 1. PQ-Shield architecture. Six server processes wrap the same FastAPI inference service: an unprotected control and five protected configurations that share the handshake/predict endpoints, the AES-256-GCM + HKDF-SHA256 session layer, and one ServerCryptoConfig contract, so only the key-establishment and signature primitives differ. ML-KEM-768 and ML-DSA-65 come from a ctypes binding to liboqs; RSA, X25519, and ECDSA from OpenSSL via the Python cryptography package.

identical signature code, $A' \rightarrow B'$ is the migration step deployed on the web today, and $B \rightarrow B'$ the price of hedging against an ML-KEM break.

Streaming experiments use a real Llama-3.2-3B-Instruct model (Q4_K_M, llama.cpp with Metal). The concurrency, payload-shape, network, session-reuse, HNDL, and MITM experiments use the non-streaming endpoint with a RandomForest classifier (or the selected payload profile). One locally hosted LLM produces ≈ 40 tokens per second for a single stream, so a 1,000-connection LLM sweep would measure the model's queue rather than the cryptography.

V. Methodology

A. Environment and Process Isolation

All measurements were taken on one host (Apple M3 MacBook Air, 8 cores, 16 GB, macOS) on AC power with low-power mode disabled, on a single day. Each configuration's server runs in its own process, is health-checked, swept, and stopped before the next configuration starts, so no two configurations share the CPU. All endpoints, including the control's, run their request handlers in the server's worker thread pool, so the control is not disadvantaged by blocking inference on the event loop. Every sweep records a run identifier, the host, and its parameters with each raw row, and the analysis uses each configuration's most recent run.

B. Cryptographic Binding and Protocol Construction

ML-KEM-768 and ML-DSA-65 are accessed through a purpose-built ctypes binding against a locally compiled liboqs [23] rather than the higher-level liboqs-python wrapper. The binding exposes exactly the six C entry points used (key generation, encapsulation, and decapsulation for ML-KEM-768; key generation, signing, and verification for ML-DSA-65), with byte lengths asserted against the FIPS 203/204 constants at every call boundary, so a build or linkage error surfaces immediately rather than propagating malformed key material. liboqs is compiled with only these two algorithms, which keeps continuous re-verification of the binding (Section VII) cheap. RSA-OAEP, X25519, ECDSA P-256, and AES-256-GCM come from OpenSSL through the Python cryptography package.

The handshake orchestration, AES-256-GCM payload encryption (keyed by HKDF-SHA256 over the raw shared secret in every configuration, including Classical-RSA's OAEP-transported key), and FastAPI plumbing are identical code paths across configurations, so differences between them can be attributed to the asymmetric primitives. By default a fresh ephemeral key pair is generated on every transaction—the conservative worst case; Section V-G measures the amortized case.

For streaming, the same signature primitive is applied to strategy-specific messages. Buffer-and-sign signs one buffered envelope (nonce || ciphertext). Per-chunk signs

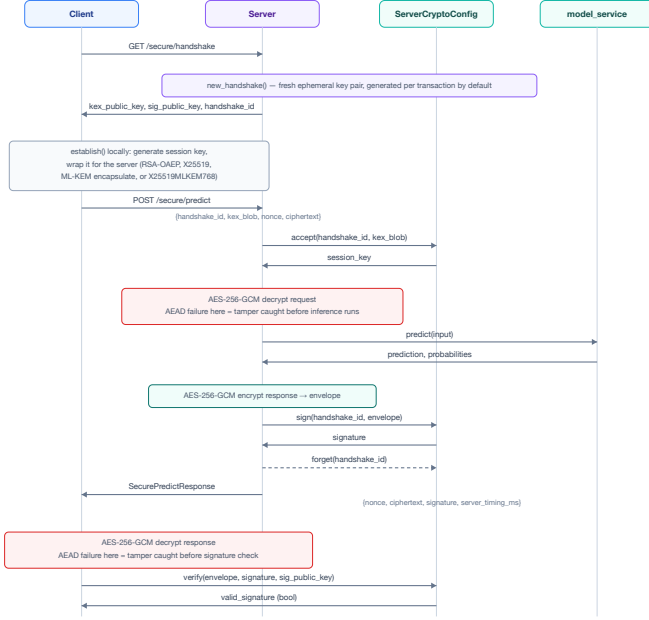


Fig. 2. One protected transaction. The client fetches fresh ephemeral keys, establishes the session key locally, and sends an encrypted request; the server decapsulates, decrypts (an AEAD failure here stops tampering before inference), infers, encrypts, and signs. The client decrypts first—an AEAD failure rejects a tampered response before any signature work—and then verifies the signature.

every chunk and prepends a 4-byte big-endian chunk index to the signed bytes, (index || nonce || ciphertext), to close the reordering vulnerability of Section III. Hash-chain folds each chunk into a running digest,

$$h_i = \text{SHA-256}(h_{i-1} \parallel \text{index}_i \parallel \text{envelope}_i), \quad (1)$$

with h_0 a 32-byte all-zero genesis value, and signs the terminal digest (and, when checkpointed, intermediate digests; Section V-F).

C. Concurrency Benchmark

Load is generated by an asynchronous client with a bounded worker pool, so the achieved concurrency matches the requested level as latency varies. Each cell (configuration $\times$ concurrency $\times$ repetition) sends ten requests per connection (with a floor for low concurrency), and five repetitions are collected at 10, 100, and 1,000 connections. The first 5% of requests in each cell are discarded as warm-up. Latency is end-to-end (total_ms: handshake round trip plus protected request), the only quantity directly comparable to the control’s single request. We report median, p95, and p99, the range of the five per-repetition medians, and the failure rate, and compare each configuration with the control at the same concurrency using the two-sided Mann–Whitney U test, which does not assume normally distributed latency. A sampler records server CPU and resident memory every 0.25 s.

D. Payload-Shape Sensitivity

Four profiles are swept at 10 connections with all six configurations and five repetitions: `tabular_small` (a 100-tree Random Forest trained on the UCI handwritten-digits dataset, 345 B request / 93 B response), `image_cnn` (a fixed-weight NumPy convolutional network on a synthetic $32 \times 32 \times 3$ image, 4,138 B / 244 B), `embedding` (a synthetic 768-dimension vector generator, 202 B / 8,022 B), and `llm_completion` (a synthetic text generator, 163 B request). The synthetic profiles probe payload shape and size; they are not claims of representative model behaviour.

E. Streaming Signature Strategies

The streaming endpoint sends chunks as Server-Sent Events over one HTTP response; each event carries one strategy-specific record (nonce, ciphertext, and, depending on strategy, a signature and/or chain hash). The model’s blocking token generator is bridged into the server’s event loop through a worker thread so that generation does not stall other requests. Time to first token (TTFT) is measured at the client from issuing the request to receiving the first event. Each configuration $\times$ strategy $\times$ length cell (50 and 200 tokens, 5-token chunks) has ten repetitions; after each server start one discarded generation absorbs the model’s cold start, and the order of cells is shuffled with a fixed seed so that no strategy is always measured first.

F. Checkpointed Hash Chains and Exposure

The hash-chain strategy can additionally sign the running digest h_i every k chunks. The client verifies each checkpoint signature against its own recomputed h_i , so any dropped, reordered, or altered chunk before a checkpoint makes that checkpoint fail mid-stream. For an n -chunk response this costs $\lfloor n/k \rfloor + 1$ signatures (checkpoints plus the terminal one), and a decrypted chunk waits at most k chunks for coverage. We record two exposure measures per transaction: the largest number of chunks displayed but not yet covered by a verified signature (max unverified chunks), and the longest time any chunk waited for coverage (verification lag). Per-chunk signing is the $k=1$ endpoint (exposure one chunk) and plain hash-chain the $k=\infty$ endpoint (exposure the whole response). We sweep $k \in \{1, 2, 5, 10, 20, \infty\}$ for Hybrid (71 B signatures) and Full PQC (3,309 B) on 200-token Llama responses in 5-token chunks (five repetitions), and at each k repeat the drop and reorder attacks, recording how much of the stream is delivered before detection.

G. Session Resumption

To measure the amortized case, a client may set `keep_session`: the server then caches the derived session key under the handshake identifier, later requests carry no key-establishment blob and reuse it, and the last request of the session ends it. Each of 10 concurrent clients reuses

one key exchange for $N \in \{1, 10, 100\}$ requests (1,000 requests per cell, five repetitions), and we report end-to-end overhead against the control.

H. Network Emulation

All other experiments run over loopback, where moving an extra 1,088-byte ML-KEM ciphertext or 3,309-byte ML-DSA signature costs almost nothing. To expose the cost of message size we place a TCP proxy between load generator and server that adds a one-way propagation delay of $RTT/2$ in each direction and a per-direction bottleneck bandwidth shared by all connections; bytes are released in order, so HTTP semantics are unchanged. We evaluate four paths—loopback, metro (10 ms RTT, 100 Mbit/s), WAN (50 ms, 20 Mbit/s), and mobile (100 ms, 5 Mbit/s)—each with all six configurations at 10 connections and five repetitions. The proxy shapes the byte stream above TCP; it does not model slow start, the initial congestion window, loss, or fragmentation, all of which penalize large handshakes further [24], so the measured cost is a lower bound.

I. Threat Injection

For HNDL, the server reports, behind a debug header that does not alter ordinary traffic, the exact size of each cryptographic artifact it sends (1,000 transactions per configuration). We distinguish harvestable bytes (everything a passive adversary captures, signatures included) from decryptable bytes (ciphertext whose session key a CRQC could recover); signatures and chain hashes are harvestable but never decryptable. For streaming, response length is varied with the configuration and strategy held fixed, so exposure can be attributed to length alone.

For MITM, a minimal forward proxy flips one bit in either the ciphertext or the signature field of each response (100 responses per configuration and target). The client decrypts before verifying, and detection time is the duration of the rejecting step itself—the AEAD tag check or the signature verification—so the two defence layers are directly comparable. Sequence attacks drop or swap a mid-stream chunk of a real streamed response and replay the sequence through the client’s incremental verification logic (ten trials per configuration, strategy, and attack).

J. Ground-Truth Validation

Two independent validations correspond to the two ways this paper’s results could be wrong. First, the primitive bindings are checked against official NIST ACVP known-answer tests [18] by re-deriving each vector’s output from its input and comparing byte for byte. Second, because no external dataset exists for streaming signature overhead, the streaming harness is checked against an analytic model: signature count and bytes follow deterministically from the number of chunks and the strategy (one signature for buffer-and-sign and plain hash-chain, one per chunk for per-chunk, $\lfloor n/k \rfloor + 1$ for checkpointed hash-chain), and

signing time follows from the count and the independently measured per-signature cost.

VI. Results

A. Cost of Protection Under Concurrency

Table III and Fig. 3 summarize the sweep. At 10 connections every configuration except Classical-RSA sits within a few milliseconds of the control (median 91 ms): Classical-ECDHE +4.9%, Hybrid +4.4%, Full PQC +3.3%, and Hybrid-KEX +7.8% (all $p < 10^{-3}$). Replacing X25519 with ML-KEM-768 therefore costs nothing measurable; the remaining few percent are the extra handshake round trip, common to all protected configurations. Classical-RSA is the outlier at +86.5% (median 169 ms), and the latency decomposition (Fig. 4) places almost all of it in the handshake, where the server generates a fresh RSA-2048 key pair.

At 100 connections the per-repetition medians of the non-RSA protected configurations are bimodal—each repetition settles near either $\approx 1,000$ ms or $\approx 1,550$ ms (e.g. Classical-ECDHE 1,014–1,701 ms, Full PQC 988–1,604 ms)—a server-queueing regime effect that dwarfs any cryptographic difference. We therefore do not rank these configurations at 100 connections; only Classical-RSA (+152.6%) is clearly separated. At 1,000 connections the four non-RSA protected configurations are indistinguishable (medians 19.7–20.5 s, +108.6–117.6% versus control), Classical-RSA reaches 30.7 s (+225.9%), and failures stay at 1.2–1.6%. That the non-RSA configurations cost about twice the control regardless of algorithm points to the second HTTP request per transaction rather than to the cryptography; Section VI-E confirms that removing the per-request handshake removes the cost.

RSA key generation is CPU-bound: In preliminary runs on a single-core host, Classical-RSA failed every request at 1,000 connections. On the 8-core host it failed only 1.5%, while consuming 3.5–4.8 cores of server CPU (Table IV) against 0.6–0.9 for every other protected configuration. Per-handshake RSA key generation therefore collapses when cores run out and survives when they do not, at several times the compute of any alternative. The primitive benchmark explains why: RSA-2048 key generation runs at 16 operations/s (median 62.7 ms over 200 samples) against 98,357/s for ML-KEM-768, a 6,167 $\times$ gap (Table X, Fig. 12).

B. Server Resources

Apart from Classical-RSA, CPU and memory are flat across configurations: at 10 connections every non-RSA configuration, including the control, uses 0.92–0.94 cores and 231–241 MB of resident memory (Table IV). Post-quantum key and signature sizes add no measurable memory footprint at this scale.

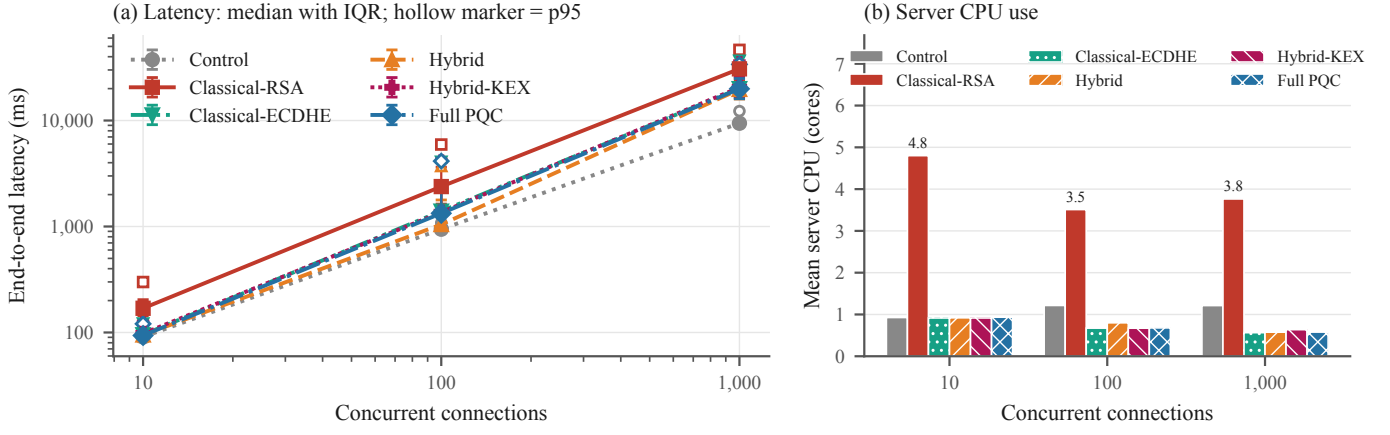


Fig. 3. Concurrency sweep. (a) End-to-end latency (median with interquartile range; hollow markers: p95) on log axes. The four non-RSA protected configurations overlap at every level; Classical-RSA is consistently slowest. (b) Mean server CPU in cores (100% = one core): per-handshake RSA-2048 key generation occupies 3.5–4.8 cores, every other configuration about one core or less.

TABLE III

End-to-end latency (handshake + protected request) by configuration and concurrency. Statistics over successful requests after 5% warm-up trimming; “Rep. range” is the spread of the five per-repetition medians. Overhead and p are versus Control at the same concurrency (two-sided Mann–Whitney U).

Conc.	Configuration	Reps	OK / attempted	Fail	Median (ms)	Rep. range (ms)	p95 (ms)	p99 (ms)	Overhead	p
10	Control	5	475 / 475	0%	90.7	89.1–92.3	117.0	129.9	ref.	–
	Classical-RSA	5	475 / 475	0%	169.2	158.2–185.7	299.3	364.5	+87%	$< 10^{-135}$
	Classical-ECDHE	5	475 / 475	0%	95.2	93.7–96.8	121.6	135.4	+5%	$< 10^{-6}$
	Hybrid	5	475 / 475	0%	94.7	93.3–96.2	117.5	134.1	+4%	$< 10^{-5}$
	Hybrid-KEX	5	475 / 475	0%	97.8	96.7–99.8	123.4	131.8	+8%	$< 10^{-12}$
	Full PQC	5	475 / 475	0%	93.7	93.0–94.2	120.6	140.4	+3%	$< 10^{-3}$
100	Control	5	4,750 / 4,750	0%	942.9	922.8–976.1	1,084.1	1,175.0	ref.	–
	Classical-RSA	5	4,745 / 4,750	0%	2,381.4	2,066.9–2,592.8	5,935.1	7,242.4	+153%	$< 10^{-299}$
	Classical-ECDHE	5	4,746 / 4,750	0%	1,396.2	1,014.0–1,701.4	4,081.1	5,527.6	+48%	$< 10^{-299}$
	Hybrid	5	4,748 / 4,750	0%	1,043.7	983.5–1,590.4	3,719.4	5,082.7	+11%	$< 10^{-236}$
	Hybrid-KEX	5	4,742 / 4,750	0%	1,361.7	1,115.1–1,638.5	4,179.9	5,510.9	+44%	$< 10^{-299}$
	Full PQC	5	4,745 / 4,750	0%	1,331.9	987.8–1,604.3	4,139.3	5,498.3	+41%	$< 10^{-299}$
1,000	Control	5	47,500 / 47,500	0%	9,428.0	9,402.4–9,586.0	12.27 s	14.27 s	ref.	–
	Classical-RSA	5	46,793 / 47,500	1%	30.73 s	29.94 s–31.23 s	46.42 s	61.57 s	+226%	$< 10^{-299}$
	Classical-ECDHE	5	46,805 / 47,500	1%	20.00 s	19.57 s–20.37 s	36.62 s	46.73 s	+112%	$< 10^{-299}$
	Hybrid	5	46,796 / 47,500	1%	19.66 s	19.09 s–20.10 s	34.23 s	41.96 s	+109%	$< 10^{-299}$
	Hybrid-KEX	5	46,907 / 47,500	1%	20.52 s	19.41 s–22.53 s	35.41 s	45.11 s	+118%	$< 10^{-299}$
	Full PQC	5	46,740 / 47,500	2%	19.92 s	19.68 s–20.23 s	33.88 s	41.73 s	+111%	$< 10^{-299}$

C. Payload Shape

Across the four profiles the cryptographic overhead is roughly constant in absolute terms (Fig. 5, Table V): +4.0–9.4 ms for every non-RSA configuration and +71.6–110.2 ms for Classical-RSA. In relative terms it ranges from +4.5% for the 89 ms RandomForest to up to +240% for millisecond-scale synthetic profiles. The cost of protection is thus an additive term independent of payload size; whether it matters depends on the inference time it is added to. Configurations using X25519 (Classical-ECDHE, Hybrid-KEX) are consistently 1.6–3.5 ms slower than those using ML-KEM alone, matching the primitive costs: server-side handshake work is 217.9 μ s for X25519 key generation plus exchange against 23.3 μ s for ML-KEM key generation plus decapsulation in our implementation.

D. Network Conditions

On emulated paths (Fig. 6) the handshake leg isolates the cost of key-exchange size. Relative to Classical-ECDHE, the ML-KEM configurations add $+4.6 \pm 0.5$ ms (Hybrid), $+4.7 \pm 0.3$ ms (Full PQC), and $+5.7 \pm 1.3$ ms (Hybrid-KEX) on the mobile path, and at most 1.0 ms on the metro and WAN paths. End-to-end, these differences are smaller than the between-repetition variability of server queueing (standard deviation 15–26 ms on WAN and mobile), so they are not resolvable at the application level. Classical-RSA’s handshake is 61–81 ms slower than Classical-ECDHE’s on every path because of key generation, not bytes. Because the proxy does not model TCP slow start or loss, these are lower bounds.

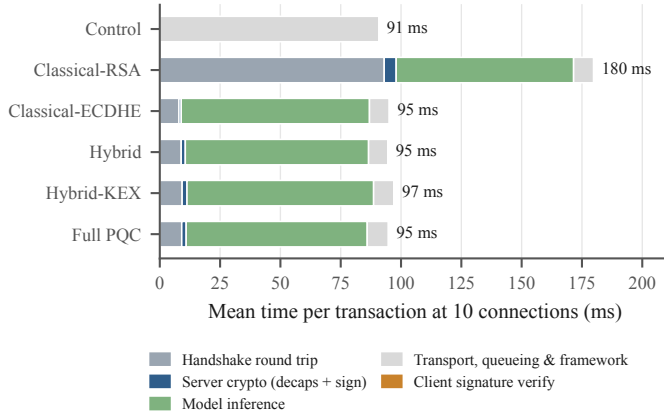


Fig. 4. Mean time per transaction at 10 connections, decomposed. Classical-RSA’s extra cost sits in the handshake (server-side RSA-2048 key generation); for the other configurations the handshake is ≈ 8 ms and model inference dominates.

TABLE IV

Server-process resource use, sampled every 0.25 s during each cell and averaged over repetitions; RSS is mean resident memory (macOS, Apple M3, 8 logical CPUs; 100% = one core).

Conc.	Config.	CPU mean (%)	CPU max (%)	RSS (MB)	Req/s
10	Control	93	126	231	107.0
	Classical-RSA	480	699	238	52.4
	Classical-ECDHE	92	125	238	101.6
	Hybrid	92	126	239	102.1
	Hybrid-KEX	92	125	241	99.4
	Full PQC	94	127	241	102.1
100	Control	122	128	241	104.8
	Classical-RSA	351	792	247	35.8
	Classical-ECDHE	67	134	245	57.8
	Hybrid	80	130	247	68.1
	Hybrid-KEX	67	132	249	56.7
	Full PQC	68	133	250	58.3
1,000	Control	121	129	269	102.5
	Classical-RSA	376	800	283	32.9
	Classical-ECDHE	56	137	280	46.1
	Hybrid	58	139	286	47.7
	Hybrid-KEX	64	142	294	45.8
	Full PQC	58	139	290	47.7

E. Session Reuse

Fig. 7 reports overhead versus the control as one key exchange serves N requests. Classical-RSA’s $+101.2\%$ at $N=1$ disappears at $N=10$ ($-1.1 \pm 1.0\%$). For the other configurations the overhead falls from $+4.0$ – 7.0% at $N=1$ to between -0.2 and $+2.1\%$ at $N=100$; one intermediate point (Full PQC at $N=10$, $+9.0 \pm 8.4\%$) reflects a single outlying repetition. For any deployment that reuses sessions, post-quantum key establishment costs nothing measurable.

F. Streaming Signature Strategies

With the real Llama-3.2-3B backend (Fig. 8, Table VI), buffer-and-sign delays the first token to the end of generation—about 5 s for a 200-token response in every

configuration (4,927–5,311 ms). Per-chunk and hash-chain signing both deliver the first token in 125–133 ms, a 39 – $42\times$ improvement that is independent of the key exchange. The strategies differ in signature bytes: for Full PQC a 200-token response carries 132,360 B of ML-DSA-65 signatures under per-chunk signing versus 3,309 B under hash-chain, a $40\times$ difference; with ECDSA the corresponding figures are 2,840 B and 71 B.

G. Bounded Exposure with Checkpointed Hash Chains

Fig. 9 and Table VII sweep the checkpoint interval k . Signature bytes follow the analytic model $(\lfloor n/k \rfloor + 1) \cdot |\sigma|$ exactly: for Full PQC 135,669 B (41 signatures) at $k=1$, 29,781 B at $k=5$, 9,927 B at $k=20$, and 3,309 B at $k=\infty$; for Hybrid 2,910 B to 70 B. Exposure grows as bytes fall: the client displays at most 5, 25, 100, and 200 tokens before a signature covers them, with maximum verification lags of 0 ms, about 0.5 s, 2.4 s, and 4.9 s. The attack experiments show the security effect directly. Without checkpoints, hash-chain detects every mid-stream drop and reorder (100%) but only at the end of the stream (0% mid-stream). With checkpoints every attack is caught mid-stream (100%), after 50–51% of the stream at $k \leq 2$ (the attack position itself), 59–60% at $k=5$, 72% at $k=10$, and 97–98% at $k=20$. Detection depends only on where checkpoints fall, so Hybrid and Full PQC give identical curves.

H. Harvest-Now-Decrypt-Later Exposure

A passive adversary captures 209–433 B per classical transaction and 1,265–4,503 B per post-quantum transaction (Fig. 10). Only the classical captures—RSA-OAEP and X25519 alike—become decryptable once a CRQC exists. Under streaming (Fig. 11) the decryptable volume grows linearly with response length for both classical key exchanges, at 11.4 bytes per generated token ($R^2=0.9997$, 50–585 tokens; the model ends longer requests early), while all three ML-KEM configurations remain at zero decryptable bytes at every length.

I. Active Tampering

Every tampered response was rejected in every configuration (Table VIII). A flipped ciphertext bit is caught by the AES-GCM tag in 0.007–0.008 ms (median), before the signature is examined; a flipped signature bit is caught by verification in 0.059 ms (ML-DSA-65) to 0.083 ms (ECDSA P-256). Per-chunk signing with the chunk index bound into the signed bytes detects drops and reorders at the affected chunk (100% mid-stream).

VII. Ground-Truth Validation

All 75 NIST ACVP known-answer tests pass (Table IX): ML-KEM-768 key generation, encapsulation, and decapsulation reproduce the published outputs byte for byte, and ML-DSA-65 verification returns the expected verdict on every vector. Byte-exact ML-DSA-65 key and signature reproduction is not possible through liboqs’s public API,

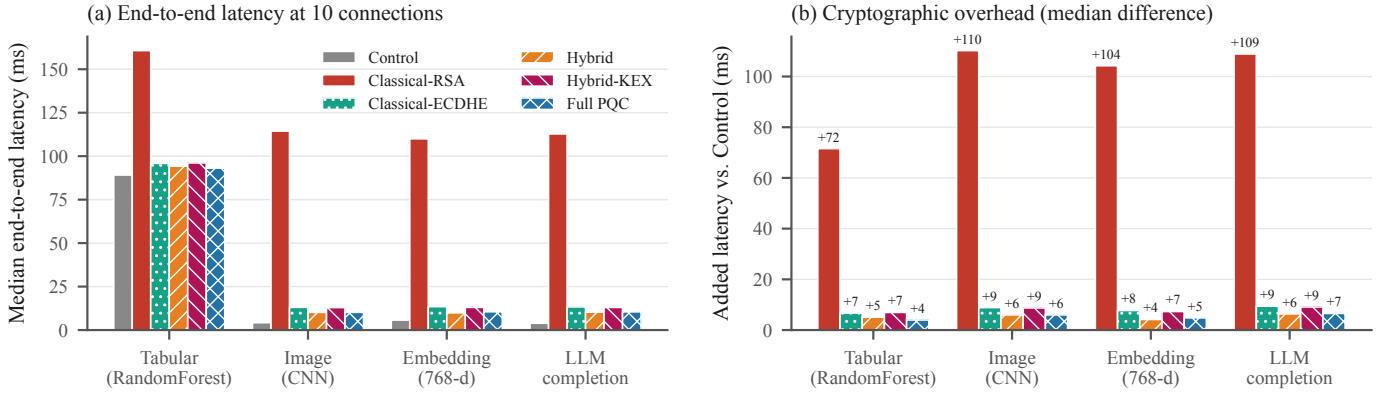


Fig. 5. Payload-shape sensitivity at 10 connections. (a) Median end-to-end latency per profile. (b) Added latency versus the control: roughly constant in milliseconds across profiles, so the cryptographic cost is additive and independent of payload size.

TABLE V

Payload-shape sensitivity at 10 connections (5 repetitions per cell). Request/response sizes are plaintext medians; each cell gives median end-to-end latency (ms) and the added latency versus Control, with Mann–Whitney U significance (* $p < 0.05$).

Profile	Req. (B)	Resp. (B)	Control	Classical-RSA	Classical-ECDHE	Hybrid	Hybrid-KEX	Full PQC
tabular_small	348	90	89.2	160.7 (+71.6*)	95.9 (+6.7*)	94.3 (+5.1*)	96.1 (+7.0*)	93.1 (+4.0*)
image_cnn	4,138	245	4.2	114.4 (+110.2*)	13.1 (+8.9*)	10.2 (+6.0*)	12.9 (+8.7*)	10.2 (+6.0*)
embedding	202	8,017	5.7	110.0 (+104.2*)	13.5 (+7.7*)	9.9 (+4.2*)	13.0 (+7.3*)	10.5 (+4.8*)
llm_completion	157	4,048	3.9	112.8 (+108.9*)	13.3 (+9.4*)	10.3 (+6.4*)	13.0 (+9.1*)	10.5 (+6.6*)

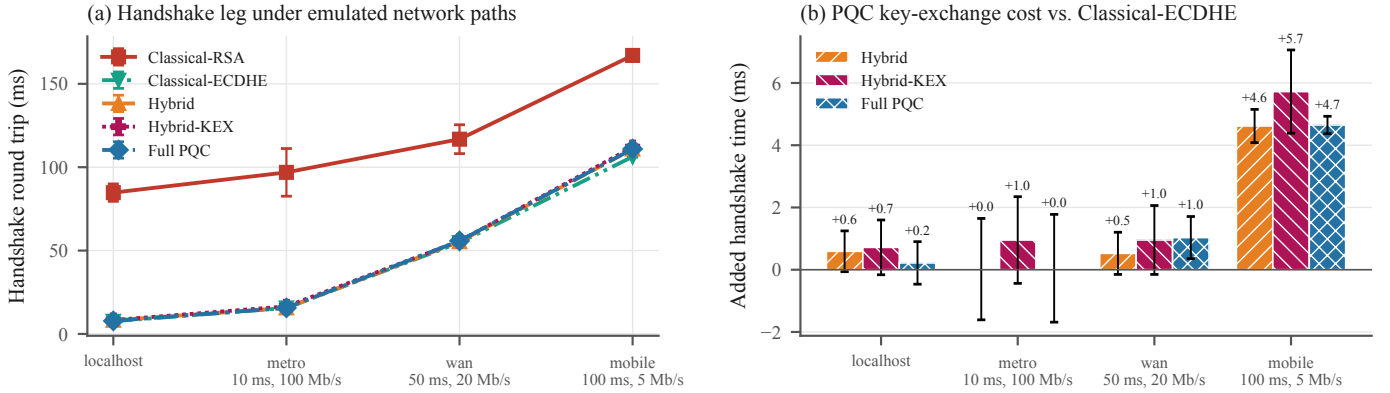


Fig. 6. Emulated network paths (10 connections; mean of five per-repetition medians, error bars one standard deviation). (a) Handshake round trip. (b) Added handshake time of the ML-KEM configurations relative to Classical-ECDHE: about 5 ms on the 5 Mbit/s mobile path, at most 1 ms elsewhere.

which exposes neither derandomized key generation nor deterministic signing.

Signature-byte predictions of the analytic model matched the harness’s measurements exactly for the three strategies and, in the checkpoint sweep, for all twelve configuration–interval cells (for example 41 ML-DSA-65 signatures, 135,669 B, at $k=1$). Timing validation surfaced a genuine effect rather than a confirmation: ECDSA P-256’s first signing call in a fresh process took 7.7–9.3 ms against a 37.8–42.3 μ s warm-loop mean, a 211–283 $\times$ gap attributable to lazy OpenSSL backend initialization, while ML-DSA-65 through our liboqs binding showed no such penalty. Correcting for this cold-start effect reduced

the two largest timing discrepancies from $281\times/297\times$ to $1.33\times/1.41\times$; a smaller residual (3–9 $\times$) remains only partially explained. Byte-overhead results are unaffected.

VIII. Discussion and Decision Framework

We construct a weighted composite score,

$$\text{composite} = w_{\text{sec}} \cdot S - w_{\text{perf}} \cdot O, \quad (2)$$

where O is the end-to-end latency overhead versus the control and S an explicit ordinal security score: 0.0 for Classical-RSA and Classical-ECDHE (no quantum resistance on either axis), 0.8 for Hybrid and Hybrid-KEX (post-quantum key exchange closes HNDL; the ECDSA signature remains classically vulnerable, a materially

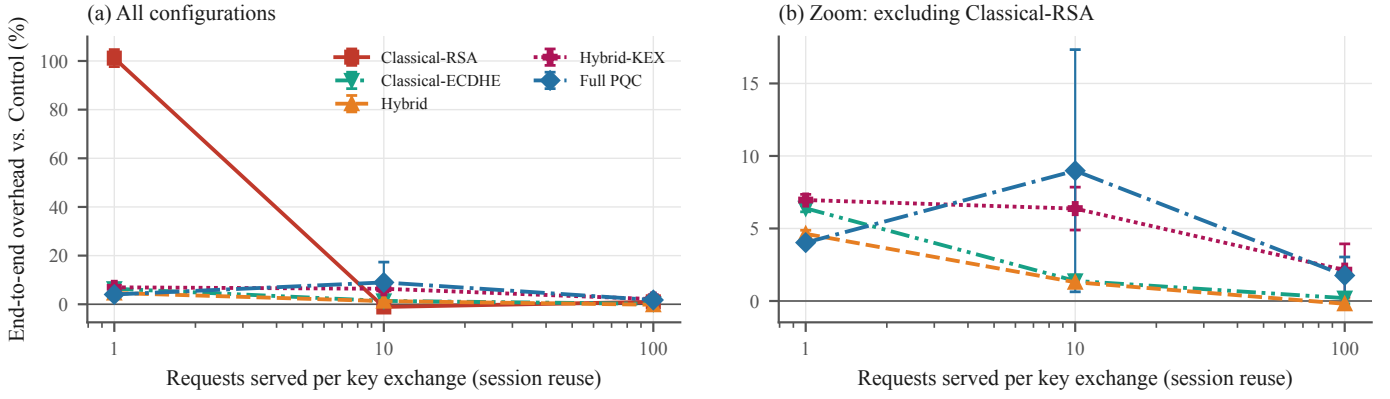


Fig. 7. Session reuse at 10 connections (mean $\pm$ one standard deviation over five repetitions). (a) All configurations: Classical-RSA’s overhead vanishes once a key exchange serves ten requests. (b) Zoom without Classical-RSA: post-quantum overhead falls to 0–2% at 100 requests per key exchange.

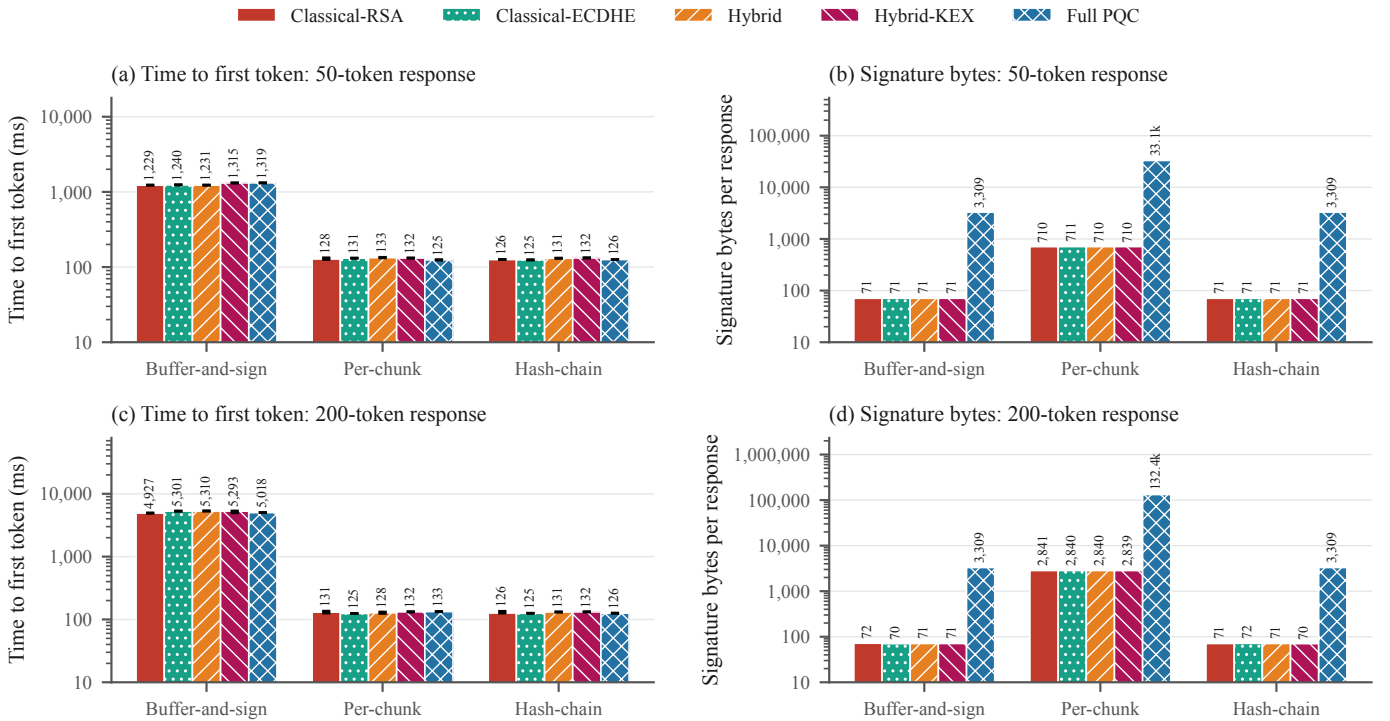


Fig. 8. Streaming signature strategies on Llama-3.2-3B (5-token chunks; median of ten repetitions, error bars interquartile range). (a, c) Time to first token: per-chunk and hash-chain deliver the first token 39–42 $\times$ sooner than buffer-and-sign. (b, d) Signature bytes per response: per-chunk ML-DSA-65 signing costs 40 $\times$ the bytes of hash-chain.

smaller risk because signatures protect real-time integrity rather than the confidentiality of harvested traffic), and 1.0 for Full PQC. Rather than collapse this judgment into one number, Table XI reports the matrix at three weightings. Full PQC scores highest in most cells because its overhead matches Hybrid’s while its security score is higher; the cells at 100 connections inherit the queueing bimodality of Section VI-A and should not be used to rank configurations.

Hypotheses: H1 (bounded post-quantum overhead) is supported: against Classical-ECDHE, moving to ML-

KEM-768 adds no measurable latency at 10 or 1,000 connections and at most 5.7ms of handshake time on the slowest emulated link; at 100 connections queueing bimodality prevents ranking. H2 (Hybrid captures Full PQC’s HNDL benefit at lower cost) is supported on HNDL—both have zero decryptable bytes—but not on cost: Full PQC is no slower than Hybrid in any non-streaming experiment and costs more only in per-chunk signature bytes. H3 (categorically different decryptability) is demonstrated: classical exposure grows linearly with streamed tokens, post-quantum exposure stays at zero. H4

TABLE VI

Streaming signature strategies on a real Llama-3.2-3B-Instruct backend (10 repetitions per cell; median, with interquartile range for TTFT). Signing and verification times are summed over the whole stream.

Tokens	Config.	Strategy	TTFT (ms)	Total (ms)	Sigs	Sig. bytes	Sign (ms)	Verify (ms)
50	Classical-RSA	Buffer-and-sign	1,229.2 [1,229–1,232]	1,230	1	71	0.07	0.07
		Per-chunk	127.9 [126–133]	1,233	10	710	0.38	0.66
		Hash-chain	125.9 [126–126]	1,230	1	71	0.08	0.07
	Classical-ECDHE	Buffer-and-sign	1,240.1 [1,237–1,244]	1,240	1	71	0.07	0.07
		Per-chunk	131.4 [131–132]	1,314	10	711	2.15	4.30
		Hash-chain	124.7 [124–125]	1,235	1	71	0.07	0.07
	Hybrid	Buffer-and-sign	1,231.1 [1,229–1,232]	1,231	1	71	0.07	0.07
		Per-chunk	133.3 [132–135]	1,327	10	710	2.24	4.36
		Hash-chain	131.0 [131–132]	1,313	1	71	0.35	0.43
	Hybrid-KEX	Buffer-and-sign	1,315.3 [1,313–1,318]	1,317	1	71	0.35	0.40
		Per-chunk	131.7 [131–132]	1,315	10	710	2.10	4.20
		Hash-chain	132.1 [132–133]	1,320	1	71	0.35	0.44
200	Classical-RSA	Buffer-and-sign	1,319.0 [1,318–1,320]	1,320	1	3,309	0.60	0.25
		Per-chunk	124.7 [124–125]	1,238	10	33,090	0.99	0.49
		Hash-chain	125.9 [126–126]	1,252	1	3,309	0.09	0.05
	Classical-ECDHE	Buffer-and-sign	4,926.9 [4,923–4,938]	4,927	1	72	0.07	0.09
		Per-chunk	130.6 [126–136]	4,933	40	2,841	1.45	2.51
		Hash-chain	126.5 [126–135]	4,921	1	71	0.07	0.07
	Hybrid	Buffer-and-sign	5,300.7 [5,297–5,305]	5,302	1	70	0.26	0.27
		Per-chunk	124.5 [124–125]	4,933	40	2,840	1.43	2.50
		Hash-chain	125.4 [125–126]	4,971	1	72	0.08	0.07
	Hybrid-KEX	Buffer-and-sign	5,309.6 [5,302–5,349]	5,311	1	71	0.25	0.27
		Per-chunk	127.9 [125–131]	5,199	40	2,840	6.89	10.58
		Hash-chain	131.3 [131–132]	5,345	1	71	0.26	0.26
	Full PQC	Buffer-and-sign	5,292.8 [4,980–5,326]	5,294	1	71	0.25	0.28
		Per-chunk	132.2 [132–133]	5,367	40	2,839	10.08	15.99
		Hash-chain	131.8 [132–132]	5,353	1	70	0.33	0.27
	Full PQC	Buffer-and-sign	5,017.9 [5,017–5,020]	5,018	1	3,309	0.13	0.09
		Per-chunk	133.4 [133–134]	5,394	40	132,360	24.32	10.74
		Hash-chain	125.7 [125–126]	5,004	1	3,309	0.11	0.05

TABLE VII

Checkpointed hash-chain signing on Llama-3.2-3B (200-token responses, 5-token chunks): a signature every k chunks plus the terminal one. Exposure = most tokens a client displays before a signature covers them (median over repetitions).

Config.	k	Sigs	Bytes	Exp. (tok)	Lag (ms)	Sign+ver. (ms)
Hybrid	1	41	2,910	5	0	4.04
	2	21	1,487	10	129	2.32
	5	9	639	25	507	1.15
	10	5	353	50	1,233	3.06
	20	3	213	100	2,419	0.38
	∞	1	70	200	4,925	0.15
Full PQC	1	41	135,669	5	0	5.54
	2	21	69,489	10	127	3.04
	5	9	29,781	25	551	8.00
	10	5	16,545	50	1,159	0.75
	20	3	9,927	100	2,426	0.44
	∞	1	3,309	200	4,911	0.14

(ML-DSA-65 verification not slower than ECDSA P-256) is confirmed: 28,202 versus 21,602 verifications/s ($1.31\times$).

IX. Recommendations

Key establishment: Migrate from X25519 to a post-quantum key exchange now: it closes HNDL exposure at no measurable latency cost, a few milliseconds on slow links, and nothing once sessions are reused. Where hedging

against an ML-KEM weakness matters, the deployed X25519MLKEM768 hybrid costs about 3ms more than ML-KEM alone at 10 connections in our implementation. Do not generate RSA keys per handshake: it occupies 4–5 CPU cores at modest load and fails when cores run out.

Signatures: ML-DSA-65 verification is faster than ECDSA P-256, so Full PQC costs no latency for single-shot responses. Its 3,309B signature matters only where many signatures are sent, which in practice means streaming.

Streaming: Use hash-chained signing with checkpoints and choose k from the exposure the application can tolerate: $k=\infty$ (one signature; the response is unverified until it ends) for low-risk display, small k where a user must not act on unverified text. With ML-DSA-65, $k=5$ bounds exposure to 25 tokens for 29,781B per 200-token response, against 132,360B for per-chunk signing. Buffer-and-sign is justified only when chunk-count and timing metadata must be hidden [29], at a $39\text{--}42\times$ time-to-first-token penalty.

X. Limitations

- Single host. All measurements come from one machine (Apple M3, 8 cores, 16 GB). The load generator and server share its CPU, which matters most at 1,000

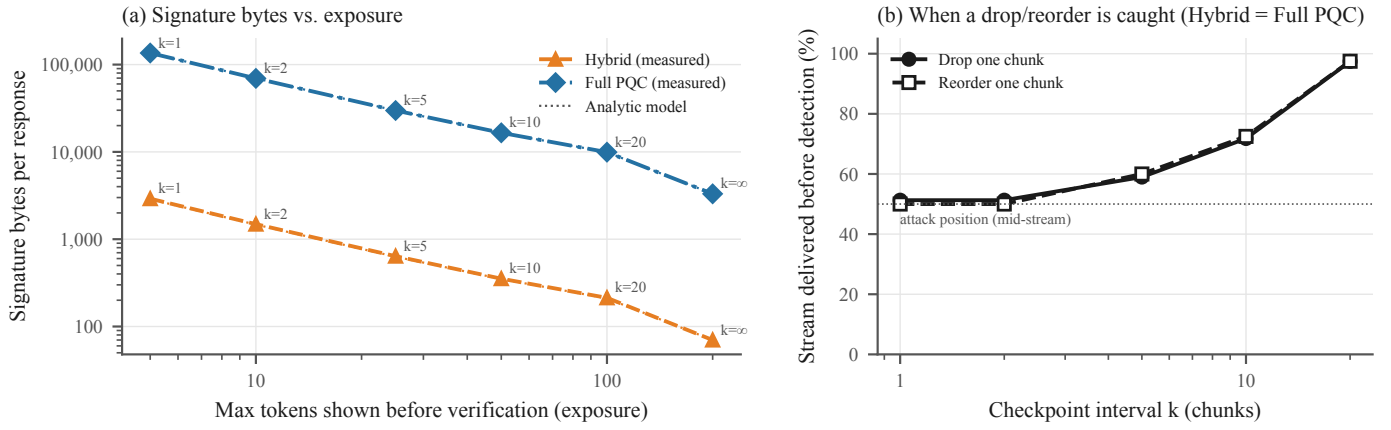


Fig. 9. Checkpointed hash chains on Llama-3.2-3B (200-token responses, 5-token chunks). (a) Signature bytes against exposure (most tokens displayed before verification); the measurements lie on the analytic model. (b) Share of the stream delivered before a mid-stream drop or reorder is detected; the dotted line marks the attack position.

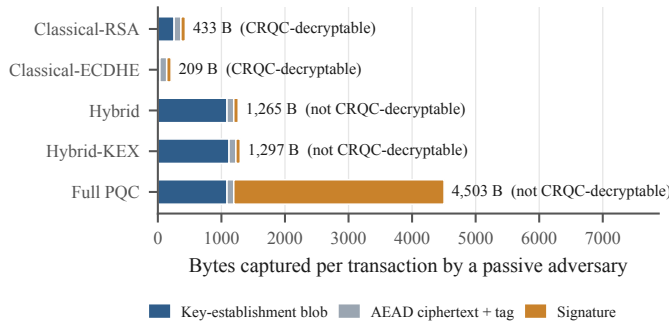


Fig. 10. Bytes a passive adversary captures per transaction, by component. Larger post-quantum captures are not decryptable by a future CRQC; the smaller classical ones are.

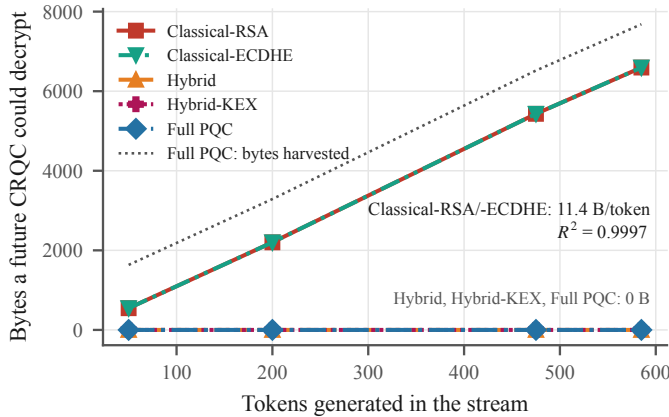


Fig. 11. Streaming HNDL exposure on Llama-3.2-3B. Under classical key exchange (RSA or X25519) the bytes a future CRQC could decrypt grow by 11.4 per generated token; under every ML-KEM configuration they stay at zero although more bytes are harvested.

connections. Relative comparisons between configurations are the intended result; absolute latencies are host-specific.

- Application-layer protocol, not TLS. PQ-Shield's

handshake is a purpose-built protocol. Its default fresh key exchange per transaction isolates the asymmetric primitive and is a worst case; Section V-G measures the amortized case, but not TLS's own resumption mechanisms (PSK, 0-RTT). Server keys are not certificate-authenticated, so it is not a substitute for authenticated TLS.

- Network emulation is byte-level. The proxy models propagation delay and bandwidth but not TCP congestion control, loss, or MTU fragmentation. Results are a lower bound on post-quantum network cost; packet-level studies [24], [25] remain the reference for lossy links.
- Classical-RSA is a worst case. Its per-handshake RSA-2048 key generation does not reflect production practice; its failure under load is a property of that design, contrasted with the realistic Classical-ECDHE baseline.
- Hybrid-KEX measured separately. Configuration B' was added after the main concurrency sweep; its concurrency results come from a supplementary run on the same host and day. Its streaming behaviour was measured with the others; its signature path is identical to Hybrid's.
- Workloads. The image_cnn profile uses an untrained fixed-weight network and the embedding/llm_completion profiles are synthetic; they probe payload shape and size, not model behaviour. Streaming uses one quantized 3B-parameter model, which may end a stream before the requested length, so streaming HNDL is reported against tokens actually generated.
- Queuing at 100 connections. Bimodal server-queuing behaviour at 100 connections prevents ranking the non-RSA configurations at that level.
- Timing-validation residual. A 3–9× gap between predicted and measured streaming signing time for some

TABLE VIII

Active tampering. (a) One flipped bit per response; detection time is the rejecting check alone. (b) Mid-stream sequence attacks on 60-token streams: share of trials detected at all, and before the stream completed (buffer-and-sign has no intermediate chunks to attack).

(a) Single-response tampering		n	Detected	Median detection (ms)	
Config.	Target				
Classical-RSA	ciphertext	100	100%	0.007	
Classical-RSA	signature	100	100%	0.063	
Classical-ECDHE	ciphertext	100	100%	0.008	
Classical-ECDHE	signature	100	100%	0.083	
Hybrid	ciphertext	100	100%	0.008	
Hybrid	signature	100	100%	0.083	
Hybrid-KEX	ciphertext	100	100%	0.008	
Hybrid-KEX	signature	100	100%	0.082	
Full PQC	ciphertext	100	100%	0.008	
Full PQC	signature	100	100%	0.059	

(b) Streaming sequence attacks					
Config.	Strategy / attack	n	Detected	Mid-stream	
Classical-RSA	Per-chunk / drop	10	100%		100%
Classical-RSA	Per-chunk / reorder	10	100%		100%
Classical-RSA	Hash-chain / drop	10	100%		0%
Classical-RSA	Hash-chain / reorder	10	100%		0%
Classical-ECDHE	Per-chunk / drop	10	100%		100%
Classical-ECDHE	Per-chunk / reorder	10	100%		100%
Classical-ECDHE	Hash-chain / drop	10	100%		0%
Classical-ECDHE	Hash-chain / reorder	10	100%		0%
Hybrid	Per-chunk / drop	10	100%		100%
Hybrid	Per-chunk / reorder	10	100%		100%
Hybrid	Hash-chain / drop	10	100%		0%
Hybrid	Hash-chain / reorder	10	100%		0%
Hybrid-KEX	Per-chunk / drop	10	100%		100%
Hybrid-KEX	Per-chunk / reorder	10	100%		100%
Hybrid-KEX	Hash-chain / drop	10	100%		0%
Hybrid-KEX	Hash-chain / reorder	10	100%		0%
Full PQC	Per-chunk / drop	10	100%		100%
Full PQC	Per-chunk / reorder	10	100%		100%
Full PQC	Hash-chain / drop	10	100%		0%
Full PQC	Hash-chain / reorder	10	100%		0%

TABLE IX

NIST ACVP known-answer-test results (75/75)

Test	Result	Basis
ML-KEM-768 keygen (derand., Alg. 16)	25/25	byte-exact
ML-KEM-768 encapsulation (Alg. 17)	25/25	byte-exact
ML-KEM-768 decapsulation (incl. 5 implicit-rej.)	10/10	byte-exact
ML-DSA-65 signature verification (Alg. 3)	15/15	correct verdict

strategy-configuration pairs remains only partially explained; signature-byte predictions match exactly.

XI. Conclusion and Future Work

Measured on a real inference service rather than a generic web server, post-quantum migration is cheap where it is most urgent. Replacing X25519 with ML-KEM-768 closes harvest-now-decrypt-later exposure at no measurable latency, a few milliseconds of handshake time on slow links, and no cost at all once sessions are reused. The expensive cases lie elsewhere: per-handshake RSA key

TABLE X

Primitive-level micro-benchmark (single process, warm loop). n = timed iterations.

Algorithm	Op.	n	Median (μ s)	p99 (μ s)	ops/s
ML-KEM-768	KeyGen	2000	10.2	12.5	98,357
ML-KEM-768	Encaps	2000	11.5	14.7	86,640
ML-KEM-768	Decaps	2000	13.1	15.0	76,190
X25519	KeyGen	2000	80.7	93.5	12,390
X25519	Exchange	2000	137.2	150.1	7,288
RSA-2048	KeyGen	200	62,698.0	257,378.1	16
RSA-2048 OAEP	Encrypt	2000	24.3	27.3	41,237
RSA-2048 OAEP	Decrypt	2000	811.3	1,730.7	1,233
ML-DSA-65	KeyGen	2000	38.0	42.3	26,316
ML-DSA-65	Sign	2000	67.9	246.1	14,733
ML-DSA-65	Verify	2000	35.5	40.8	28,202
ECDSA P-256	KeyGen	2000	15.5	18.5	64,687
ECDSA P-256	Sign	2000	21.0	23.7	47,619
ECDSA P-256	Verify	2000	46.3	54.2	21,602

generation, which is CPU-bound and fails when cores run out, and per-chunk post-quantum signatures on streamed responses, which multiply signature bytes by the number of chunks. For streaming, checkpointed hash chains turn

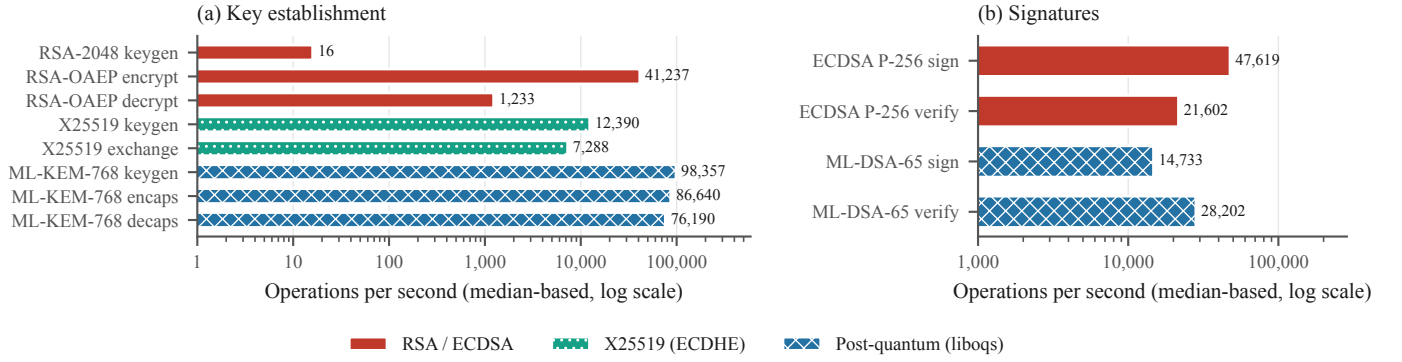


Fig. 12. Primitive throughput (single process, warm loop, median-based). (a) Key establishment, log scale: RSA-2048 key generation is $6,167\times$ slower than ML-KEM-768 key generation, and ML-KEM-768 is also cheaper than X25519. (b) Signatures: ML-DSA-65 verification is $1.31\times$ faster than ECDSA P-256 despite a $\approx 47\times$ larger signature.

TABLE XI

Weighted composite score $w_{sec} \cdot S - w_{perf} \cdot O$ (higher is better), where S is the security score (Classical 0, Hybrid 0.8, Full PQC 1.0) and O the end-to-end latency overhead versus Control. Bold: best at that weighting and concurrency; fail: every request failed; n/a: not measured.

Config.	Security ($w_{sec}=0.8$)			Balanced ($w_{sec}=0.5$)			Performance ($w_{sec}=0.2$)		
	10	100	1,000	10	100	1,000	10	100	1,000
Classical-RSA	-0.17	-0.31	-0.45	-0.43	-0.76	-1.13	-0.69	-1.22	-1.81
Classical-ECDHE	-0.01	-0.10	-0.22	-0.02	-0.24	-0.56	-0.04	-0.38	-0.90
Hybrid	+0.63	+0.62	+0.42	+0.38	+0.35	-0.14	+0.12	+0.07	-0.71
Hybrid-KEX	+0.62	+0.55	+0.40	+0.36	+0.18	-0.19	+0.10	-0.20	-0.78
Full PQC	+0.79	+0.72	+0.58	+0.48	+0.29	-0.06	+0.17	-0.13	-0.69

TABLE XII
Recommendation summary

Scenario	Recommendation
Any inference API	Key exchange: ML-KEM-768, or X25519MLKEM768 to hedge against an ML-KEM break
Non-streaming responses	ECDSA P-256 or ML-DSA-65 (no latency difference)
Streaming, display-only	Hash-chain, $k = \infty$
Streaming, actionable output	Hash-chain with small k (e.g. $k = 5$: 25-token exposure)
Long-term non-repudiation	ML-DSA-65 signatures (Full PQC)
Metadata-sensitive streaming	Buffer-and-sign
Legacy RSA key transport	Migrate away

that cost into an explicit dial between signature bytes and the amount of text a client shows before verifying it.

Future work includes multi-host and packet-level network measurements, mid-stream rekeying to bound HNDL exposure per session, TLS-native resumption, and a trained, larger-scale vision workload.

Acknowledgment

The authors thank the faculty and technical staff of the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, for the resources

and infrastructure used in this study, and the open-source cryptographic and machine-learning communities, whose tools and standards supported its implementation, validation, and evaluation.

References

- [1] M. Abbasi, F. Cardoso, P. Váz, J. Silva, and P. Martins, “A practical performance benchmark of post-quantum cryptography across heterogeneous computing environments,” *Cryptography*, vol. 9, no. 2, art. 32, 2025, doi: 10.3390/cryptography9020032.
- [2] E. D. Demir, B. Bilgin, and M. C. Onbasli, “Performance analysis and industry deployment of post-quantum cryptography algorithms,” *arXiv:2503.12952*, 2025.
- [3] O. Egbuagha and E. Ikwunna, “Post-quantum cryptography in practice: A literature review of protocol-level transitions and readiness,” *IACR Cryptology ePrint Archive*, Paper 2025/1668, 2025.
- [4] P. Pote and R. Bansode, “Performance evaluation of post-quantum cryptography: A comprehensive framework for experimental analysis,” *J. Inf. Syst. Eng. Manage.*, vol. 10, no. 9s, pp. 548–556, 2025, doi: 10.52783/jisem.v10i9s.1253.
- [5] A. Shaw, “quantum-safe: Bridging the post-quantum production gap with a hybrid-by-default Python cryptography library,” *arXiv:2605.17061*, 2026.
- [6] J. A. Montenegro, R. Rios, and J. Lopez-Cerezo, “A performance evaluation framework for post-quantum TLS,” *Future Gener. Comput. Syst.*, vol. 175, art. 108062, 2026, doi: 10.1016/j.future.2025.108062.
- [7] G. Chhetri, S. Somvanshi, P. Hebli, S. Brotee, and S. Das, “Post-quantum cryptography and quantum-safe security: A comprehensive survey,” *arXiv:2510.10436*, 2025.
- [8] R. A. Faval, R. Moreira, and F. de Oliveira Silva, “Empowering mobile networks security resilience by using post-quantum cryptography,” *arXiv:2603.28626*, 2026.

- [9] A. Kudaloor and A. Aijaz, "Toward quantum-safe 6G: Experimental evaluation of post-quantum cryptography techniques," *IEEE Commun. Standards Mag.*, accepted, 2026, arXiv:2605.06881.
- [10] L. Song, R. Shokri, and P. Mittal, "Privacy risks of securing machine learning models against adversarial examples," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 241–257.
- [11] L. N. Tidjon and F. Khomh, "Threat assessment in machine learning based systems," arXiv:2207.00091, 2022.
- [12] N. Ahmed, L. Zhang, and A. Gangopadhyay, "A survey of post-quantum cryptography support in cryptographic libraries," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2025, arXiv:2508.16078.
- [13] J. Blanco-Romero, F. Almenares Mendoza, C. García Rubio, C. Campo, and D. Díaz Sánchez, "On the practical feasibility of harvest-now, decrypt-later attacks," arXiv:2603.01091, 2026.
- [14] D. Gómez-Cambronero, D. Munteanu, and A. I. González-Tablas, "Layered performance analysis of TLS 1.3 handshakes: Classical, hybrid, and pure post-quantum key exchange," arXiv:2603.11006, 2026.
- [15] A. G. Schemitt, H. F. da Silva, R. C. Lunardi, D. Kreutz, R. B. Mansilha, and A. F. Zorzo, "Assessing the impact of post-quantum digital signature algorithms on blockchains," in *Proc. IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, 2025, arXiv:2510.09271.
- [16] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," FIPS PUB 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203.
- [17] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," FIPS PUB 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204.
- [18] National Institute of Standards and Technology, "Automated Cryptographic Validation Protocol (ACVP) server and test vectors," 2024. [Online]. Available: <https://github.com/usnistgov/ACVP-Server>
- [19] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," IETF RFC 8446, Aug. 2018.
- [20] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, "Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3," IETF Internet-Draft draft-ietf-tls-ecdhe-mlkem-05, May 2026.
- [21] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [22] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?" *IEEE Secur. Privacy*, vol. 16, no. 5, pp. 38–41, 2018.
- [23] D. Stebila and M. Mosca, "Post-quantum key exchange for the Internet and the Open Quantum Safe project," in *Selected Areas in Cryptography – SAC 2016*, ser. LNCS, vol. 10532. Springer, 2017, pp. 14–37.
- [24] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Post-Quantum Cryptography – PQCrypto 2020*, ser. LNCS, vol. 12100. Springer, 2020, pp. 72–91.
- [25] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2020.
- [26] R. Gennaro and P. Rohatgi, "How to sign digital streams," in *Advances in Cryptology – CRYPTO '97*, ser. LNCS, vol. 1294. Springer, 1997, pp. 180–197.
- [27] C. K. Wong and S. S. Lam, "Digital signatures for flows and multicasts," *IEEE/ACM Trans. Netw.*, vol. 7, no. 4, pp. 502–513, 1999.
- [28] A. Perrig, R. Canetti, J. D. Tygar, and D. Song, "Efficient authentication and signing of multicast streams over lossy channels," in *Proc. IEEE Symp. Secur. Privacy*, 2000, pp. 56–73.
- [29] R. Weiss, D. Ayzenshteyn, G. Amit, and Y. Mirsky, "What was your prompt? A remote keylogging attack on AI assistants," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 3367–3384.